

Multimedia Retrieval

Chapter 3: Performace Evaluation

Dr. Roger Weber, roger.weber@gmail.com

[3.1 Setting Up a Benchmark](#)

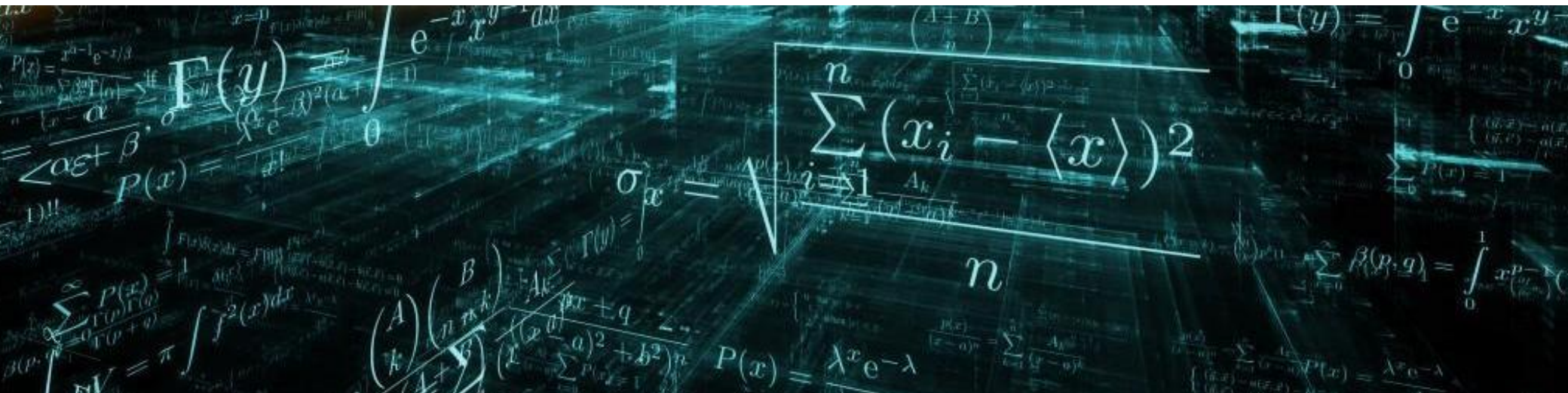
[3.2 Precision and Recall](#)

[3.3 Retrieval with Order](#)

[3.4 Retrieval with Graded Relevance](#)

[3.5 Retriever and Filter/Ranker](#)

[3.6 Literature and Links](#)

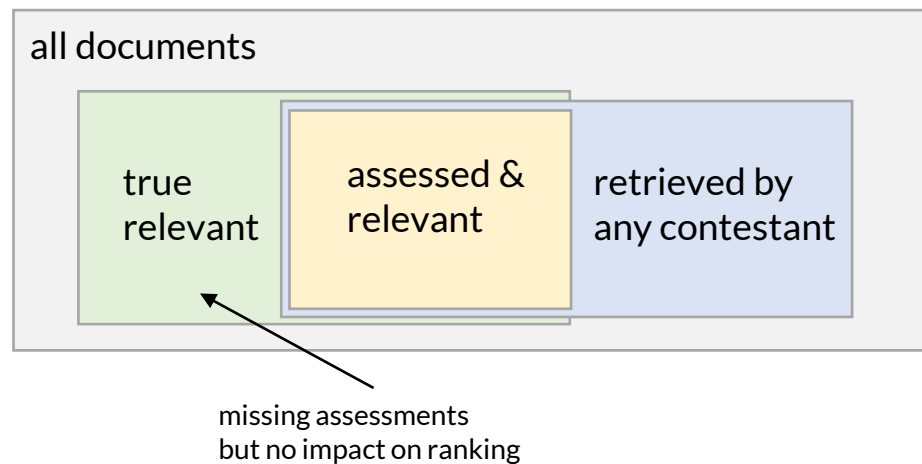


3.1 Setting Up a Benchmark

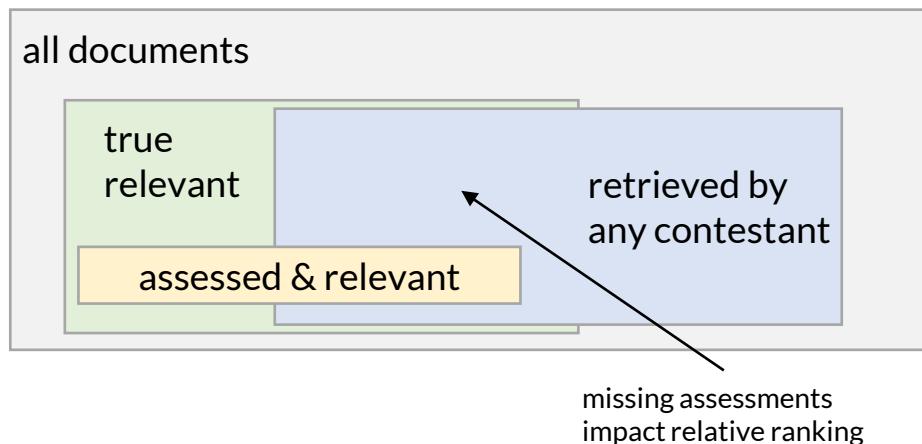
- In information retrieval, comparing methods is essential to see how well systems meet users' needs. A new ranking algorithm or neural model may look promising on its own, but without a shared test against existing approaches, claims of improvement stay anecdotal. Benchmarks give that common ground: curated collections of documents, queries, and relevance judgments that let us run the same experiments and use the same metrics. This lets us compare different approaches or fine tune hyperparameters to get better results.
- There is no single best benchmark or evaluation method. Retrieval goals vary widely, from answering fact-checking questions in milliseconds to surfacing long, nuanced reports for expert analysts. A collection built for web search will not capture the challenges of legal discovery, medical literature review, or cross-lingual retrieval. Metrics that reward precision in the top ten results may be irrelevant when recall or diversity matter most. Effective evaluation depends on the intended use case: the domain, the users, and the trade-offs they care about.
- An effective benchmark consists of four closely linked parts: a document collection, a set of queries or topics, a method for judging relevance, and clear performance goals. Each part helps create a fair testing environment for comparing retrieval methods.
 - **Document collection.** This is the foundation, the universe of data a search system must explore. It can include news articles, academic papers, legal filings, social media posts, or product descriptions, depending on the application. A good collection is stable so results remain reproducible over time, and it is representative so it reflects the real domain. Classic examples include MS MARCO, with more than half a million Bing queries and millions of passages for web style search, and the TREC collections, which cover sources from newspaper archives to biomedical abstracts. Domain specific corpora such as PubMed Central for medical literature and arXiv for scientific papers show how specialized benchmarks address particular needs.
 - **Queries or topics.** They express real users' information needs. They may be short keyword searches (“pizza”) or detailed natural-language questions (“For which research did Albert Einstein get the noble prize?”). A good query set covers simple factual lookups and complex exploratory tasks. For example, the TREC Web Track uses both brief navigational queries and multi-sentence investigative topics to show how systems handle different levels of complexity within the given use case scenario.

- **Relevance Judgments.** To measure retrieval accuracy, we mark each query-document pair as relevant or not relevant. Because relevance is subjective, assessors usually follow written guidelines to keep judgments consistent. Two main approaches are used in practice:
 - **Dense Assessments:** In traditional competitions, each contestant runs every query. Organizers pool the documents retrieved by all participants and then judge them. This produces a relatively dense set of relevance assessments that covers all documents any system retrieved. Although not every document is judged, this approach generally preserves the relative ranking of competing algorithms. For example, an unjudged relevant document may slightly inflate an algorithm's recall estimate, but including its assessment does not change the methods' comparative performance.
 - **Sparse Assessments:** Large scale benchmarks like MS MARCO cover thousands of queries, so judging every retrieved document is impractical. Organizers therefore evaluate only a subset of documents per query, leaving relevance labels sparse. Many retrieved documents remain unjudged, and some relevant items may never be evaluated. As a result, a system can retrieve genuinely relevant material but receive no credit if those documents were not judged. When interpreting results, it is important to note that sparse judgments primarily affect the evaluation of **top-ranked precision**, while metrics measuring full-recall performance may be less reliable.

Dense Assessments



Sparse Assessments



- **Performance goals.** A benchmark also defines what success looks like by listing the metrics and criteria for the use case. For example, web search often values high precision in the top ten results and low latency. Legal discovery tends to prioritize high recall. Recommendation systems usually aim for a balance of diversity and novelty. Metrics such as Mean Average Precision (MAP), Normalized Discounted Cumulative Gain (NDCG), or task specific cost models tie these goals to measurable outcomes. Examples (we define the metrics later in more details):
 - A **web search engine** should give instant answers to millions of users worldwide. Its main goal is to return highly relevant results, with top-ten precision above 85% and NDCG@10 above 0.90. Users expect near-instantaneous responses, so average query latency must stay below 200 milliseconds. The system must handle peak loads of over 10,000 queries per second and keep cost per query extremely low, ideally less than a fraction of a cent. The core challenge is balancing speed and relevance to deliver a smooth user experience.
 - In **legal document discovery**, the goal is to find every document relevant to a lawsuit or regulatory review, since missing even one relevant item can have serious consequences. Systems are judged mainly on high overall recall, meaning they must retrieve nearly all relevant documents rather than stopping at an arbitrary cutoff such as the top 100 results. Query latency is less important; queries that take minutes are acceptable, and throughput needs are moderate, often just a few queries per day. Precision matters too to avoid overwhelming reviewers with irrelevant material, but recall remains the primary success measure.
 - An **e-commerce recommendation system** delivers personalized product suggestions in real time. Success is measured by relevance, for example a high click-through rate typically above 5 percent, while keeping recommendations diverse. To keep users engaged, latency must be low and recommendations should appear within 50 milliseconds. Throughput must scale to millions of recommendations per hour to serve all active users. The system must also control costs by using efficient batch inference and caching, balancing relevance and novelty with computational efficiency.
 - For a **biomedical research portal**, the goal is to help scientists locate a substantial set of relevant scientific papers or clinical studies. Success requires a careful balance between recall and precision: researchers want to find as many relevant documents as possible, but high precision is also critical to avoid wasting time reading irrelevant papers. Performance is evaluated over a larger number of retrieved results (beyond just the top 20), with metrics like MAP or recall@100–200 used to reflect this balance. Latency should remain under one second to maintain a smooth search experience, and throughput is moderate, typically hundreds to thousands of queries per day. The emphasis is on supporting high-quality, evidence-based research by delivering comprehensive and relevant document sets efficiently.

3.1.1 Practical Aspects for Defining a Robust Benchmark

- **Document collection**

- **Purpose and scope.** The collection must match the domain or use case being evaluated, such as general web search, scientific literature, or legal documents. Results from one domain do not automatically transfer to another. When choosing the best method for an application, the benchmark must match the data and scope of that use case.
- **Reproducibility.** Keep the collection fixed during experiments. To support fine-tuning and improvements over the application lifetime, keep the benchmark stable and avoid frequent expansions. Each change requires costly reassessments of relevance and re-evaluations of retrieval results. Avoid benchmarks that target a single feature, as they tend to favor that feature. Instead, design benchmarks from the user perspective and without assuming any specific search method.
- **Representativeness.** Make sure the collection reflects realistic document distributions, content types, and topic diversity for the target users. Preprocess documents consistently: extract text, remove duplicates, assign metadata, and index each document with a unique identifier so retrieval and assessment are accurate.

- **Queries or topics**

- **Purpose and realism.** Queries express the target users' information needs. They should come from real user queries so they match those needs as closely as possible. For new document collections or when user data is scarce, large language models (LLMs) can generate extra queries based on user behavior patterns or on samples from the collection. This expands coverage, keeps queries realistic, and reduces manual work.
- **Clarity and consistency.** Each query should state a clear information need to avoid ambiguity for assessors. Ambiguous queries reduce the consistency of relevance judgments and weaken evaluation validity. When possible, check queries with pilot tests or expert review to ensure they produce meaningful result distributions and match likely user expectations.
- **Quantity.** Provide enough queries for statistically robust evaluation. Small collections may need 25 to 50 queries for basic experiments, while larger benchmarks should include 100 or more to capture topic diversity and enable meaningful comparisons across retrieval systems.
- **Maintenance and stability.** Queries, like the document collection, should stay fixed to ensure reproducible results. Changing queries requires new relevance assessments and can invalidate earlier evaluations. Record all query sources, how queries were created, and how they were validated to keep experiments transparent and repeatable.

- **Relevance Judgements**

- **Purpose and grounding.** Relevance judgments are the ground truth for evaluating retrieval systems. They indicate which documents satisfy each query's information need, and they are essential for reliably measuring system performance and making meaningful comparisons. They should not change over the lifetime of a benchmark, because any change would render previous results useless.
- **Consistency and clarity.** Human assessors must follow clear guidelines so their judgments are consistent. If instructions are unclear or relevance criteria are vague, assessors may disagree and the benchmark may lose validity. Training, practice examples, and regular quality checks help align assessors' judgments.
- **Scale and efficiency.** Assessing every document for every query is rarely possible for large collections. Methods like pooling, which means judging only the top results from competing retrieval systems, or statistical sampling reduce the workload while keeping evaluations reliable.
- **LLM-assisted assessment.** Large language models (LLMs) can create initial relevance judgments by screening documents or suggesting judgments for clear cases. Human assessors then review uncertain or borderline items. This hybrid approach improves efficiency, lowers costs, and speeds up benchmark creation, especially for new or large document collections.

- **Performance goals.**

- **Purpose and definition.** Performance goals define success for a retrieval system on a benchmark. Different domains require different performance aspects, and a metric useful in one case may be less useful in another. Precision and recall are the most common metrics, but scenario-specific definitions are often needed for meaningful results.
- **Relevance and ranking metrics.** Metrics like MAP, NDCG, and precision at k measure how well a system finds relevant documents, but the importance of ranking varies by domain. In legal search, ranking matters less. High recall and good precision across all relevant documents are critical. In web search, users focus on the top results, so ranking is most important. Biomedical literature search needs a balance of recall and precision over a larger set to provide comprehensive but manageable results.
- **Operational metrics.** Beyond relevance, practical performance factors include latency, throughput, and cost. Latency is the system's response time, usually measured in milliseconds or seconds, and it affects user satisfaction. Throughput is how many queries or recommendations the system can handle per unit of time, which matters in high-traffic settings. Cost covers the computing resources needed for retrieval.

3.2 Precision and Recall

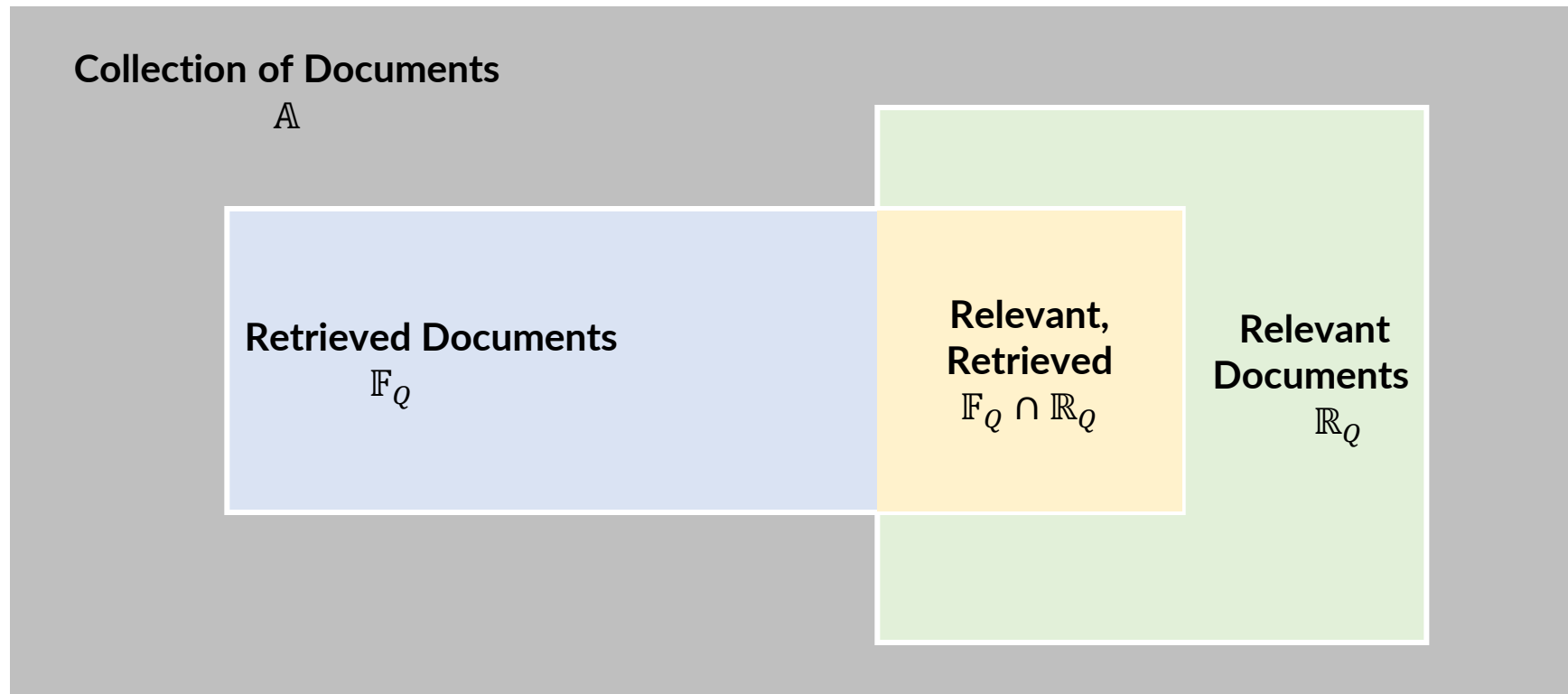
- First, we examine Boolean retrieval. In this simple case, documents are returned as an unordered set. The system gives a list of documents, but we do not consider their order when we evaluate it. Later, we will explore how to extend this approach to take document order into account.
- **Precision** and **recall** are the earliest and still most important measures used for the evaluation of search algorithms. Precision denotes how many of the answers are relevant from a user's perspective. Recall describes the percentage of retrieved and relevant answers over all relevant documents in the collection. They form the key dimensions that covers the user's interests:
 - **Precision** measures how easily relevant documents can be found without forcing users to sift through many results. Often, users need only a few documents that provide the required information, not every relevant item. A search engine that returns mostly relevant documents is well suited to knowledge tasks and fact checking, such as student research or professional fact-checking.
 - **Recall** is about finding most of the relevant documents. In these cases we want a comprehensive view of those documents and we want to keep the number of non-relevant items low (false positives). A good search engine retrieves most or all relevant documents while minimizing non-relevant results.
- Notations:

$\mathbb{A}$	Set of all documents
$\mathbb{R}_Q$	Set of relevant documents for a query Q in the collection $\mathbb{A}$
$\mathbb{F}_Q$	Set of documents retrieved by a system for query Q

- Precision p , recall r , and fallout f are then defined as (see visualization on next page):

$$p = \frac{|\mathbb{F}_Q \cap \mathbb{R}_Q|}{|\mathbb{F}_Q|} \qquad r = \frac{|\mathbb{F}_Q \cap \mathbb{R}_Q|}{|\mathbb{R}_Q|} \qquad f = \frac{|\mathbb{F}_Q \setminus \mathbb{R}_Q|}{|\mathbb{A} \setminus \mathbb{R}_Q|}$$

- Visualization of precision and recall

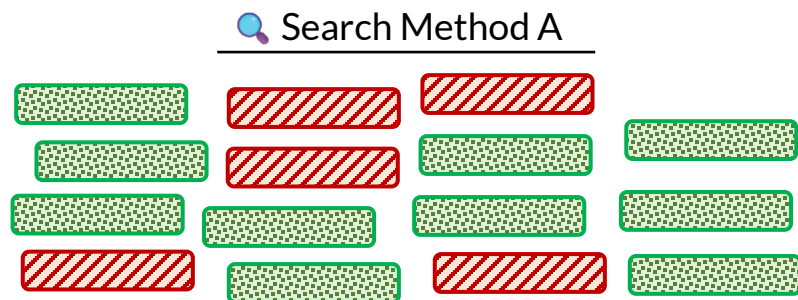


Precision: $p = \frac{|\mathbb{F}_Q \cap \mathbb{R}_Q|}{|\mathbb{F}_Q|}$

Recall: $r = \frac{|\mathbb{F}_Q \cap \mathbb{R}_Q|}{|\mathbb{R}_Q|}$

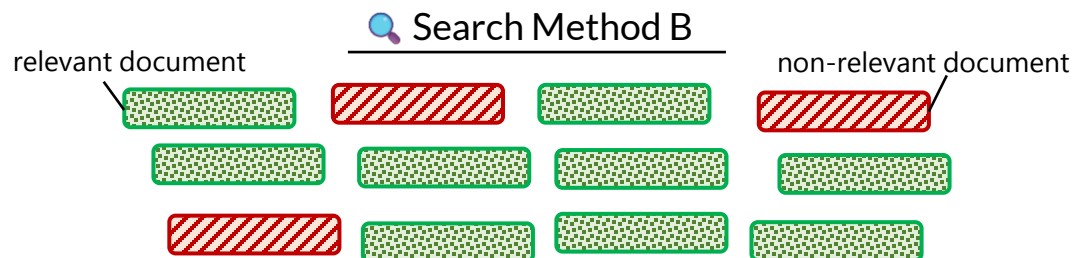
Fallout: $f = \frac{|\mathbb{F}_Q \setminus \mathbb{R}_Q|}{|\mathbb{A} \setminus \mathbb{R}_Q|}$

- To better understand these metrics, we introduce some foundational related terms:
 - **True Positives (TP):** Relevant documents that the system correctly retrieves.
 - **False Positives (FP):** Non-relevant documents that the system incorrectly retrieves.
 - **False Negatives (FN):** Relevant documents that the system fails to retrieve.
 - **True Negatives (TN):** Non-relevant documents that the system correctly does not retrieve.
 - **Fallout (f):** The proportion of non-relevant documents retrieved out of all non-relevant documents in the collection. It helps measure how much “noise” the system produces.
- Let’s illustrate these concepts with a concrete example. Suppose a query has **20 relevant documents** in a collection of 1000 documents. Two different search methods are applied:



$$TP=10, FP=5, FN=10, TN=975$$

$$p = \frac{10}{15} = 67\% \quad r = \frac{10}{20} = 50\%$$



$$TP=9, FP=3, FN=11, TN=977$$

$$p = \frac{9}{12} = 75\% \quad r = \frac{9}{20} = 45\%$$

• Comparison of Method A and Method B:

- **Precision:** Method B (75%) is higher than Method A (67%). This means Method B returns a higher proportion of relevant documents, making it more efficient for users who want fewer irrelevant results.
- **Recall:** Method A (50%) is slightly higher than Method B (45%). This indicates Method A retrieves more of the total relevant documents, which is useful when comprehensiveness is important.
- **Trade-off:** Method A sacrifices precision for better recall, while Method B sacrifices recall to improve precision.
- **Recommendation:** If the goal is to quickly find mostly relevant documents (e.g., fact-checking or quick research), **Method B** is preferable due to higher precision. If the goal is to find as many relevant documents as possible (e.g., exhaustive research or legal searches), **Method A** is preferable due to higher recall.

- The **F-Measure** combines precision and recall into a single value, simplifying the comparison of different retrieval methods. The parameter β determines the importance of recall over precision. When $\beta=0$, only precision is considered; when $\beta=\infty$, only recall is considered.

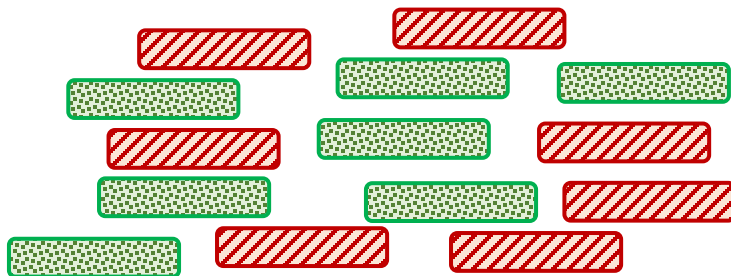
$$F_{\beta} = \frac{(\beta^2 + 1) \cdot p \cdot r}{\beta^2 \cdot p + r}$$

$$F_1 = \frac{2 \cdot p \cdot r}{p + r} = \text{F-score}$$

The **F_1 -score** is a common choice with $\beta=1$ and is also frequently used in machine learning tasks to optimize hyperparameters (see later in this course). Generally, β should be selected thoughtfully depending on the retrieval task's performance goal. For example, for fact-checking tasks, precision is prioritized over recall, making a smaller $\beta=0.5$ suitable. On the other hand, a patent lawyer may choose $\beta=2$ to emphasize the importance of retrieving many relevant documents while maintaining reasonable precision.

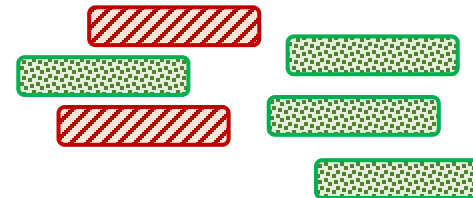
- Example: Comparing two methods (query has a total of 20 relevant documents)

 Search Method A

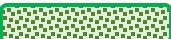


$$p = \frac{7}{14} = 50\% \quad r = \frac{7}{20} = 35\% \quad F_1 = \frac{2 \cdot \frac{7}{14} \cdot \frac{7}{20}}{\frac{7}{14} + \frac{7}{20}} = 0.41$$

 Search Method B



$$p = \frac{4}{6} = 67\% \quad r = \frac{4}{20} = 20\% \quad F_1 = \frac{2 \cdot \frac{4}{6} \cdot \frac{4}{20}}{\frac{4}{6} + \frac{4}{20}} = 0.31$$

relevant document  non-relevant document 

- Typically, we have multiple queries, and for each query, we calculate a precision-recall pair. To evaluate the overall retrieval performance, we need to compute average precision and recall. Let N represent the number of queries. For each query Q_i , we have a set $\mathbb{F}_i$ retrieved by the search method and a set $\mathbb{R}_i$ of relevant documents for that query. We use p_i and r_i to denote the precision and recall values, respectively, as explained earlier.
- With **macro evaluation**, we simply compute the average over all precision and recall values as follows:

$$p = \frac{1}{N} \sum_{i=1}^N p_i = \frac{1}{N} \sum_{i=1}^N \frac{|\mathbb{F}_i \cap \mathbb{R}_i|}{|\mathbb{F}_i|} \qquad r = \frac{1}{N} \sum_{i=1}^N r_i = \frac{1}{N} \sum_{i=1}^N \frac{|\mathbb{F}_i \cap \mathbb{R}_i|}{|\mathbb{R}_i|}$$

While the macro evaluation method is generally effective, it can have limitations when dealing with varying sizes of relevant documents for queries. For example, consider a query Q_i that has only one relevant document in the entire collection while the other queries have hundreds of relevant documents. Not finding that relevant document for Q_i would result in a precision value $p_i = 0$. This can significantly lower the average precision, even if the method performs well and produces high precision values for the other queries.

- **Micro evaluation** overcomes this issue by summing the true positives and the retrieved/relevant documents before calculating the average precision and recall. This ensures a fair evaluation regardless of the result sizes of queries:

$$p = \frac{\sum_{i=1}^N |\mathbb{F}_i \cap \mathbb{R}_i|}{\sum_{i=1}^N |\mathbb{F}_i|} \qquad r = \frac{\sum_{i=1}^N |\mathbb{F}_i \cap \mathbb{R}_i|}{\sum_{i=1}^N |\mathbb{R}_i|}$$

With the example from above, the impact on missing out on the relevant document Q_i is now much smaller and may better suit the retrieval benchmark's design.

- The choice between micro and macro evaluation depends on the nature of the retrieval task and the importance given to different queries. Micro evaluation tends to emphasize the performance on queries with more relevant documents since they contribute more to the overall counts. Macro evaluation, on the other hand, gives equal weight to each query, regardless of its size or importance, providing a more balanced view of the overall performance across all queries.

- Example:

Query	Relevant Docs ($ R_i $)	Retrieved Docs ($ F_i $)	True Positives (TP)	Precision (p_i)	Recall (r_i)
Q1	1	1	0	0.0	0.0
Q2	100	50	40	0.8	0.4
Q3	50	50	25	0.5	0.5

- **Macro Evaluation**

Macro Precision: $(0.0+0.8+0.5)/3 = 0.433$

Macro Recall: $(0.0+0.4+0.5)/3 = 0.3$

Interpretation: Each query contributes equally, so Q1 (with only 1 relevant doc missed) heavily lowers the overall averages.

- **Micro Evaluation**

- Sum of TP: $0 + 40 + 25 = 65$
- Sum of Retrieved Docs: $1 + 50 + 50 = 101$
- Sum of Relevant Docs: $1 + 100 + 50 = 151$

Micro Precision: $65 / 101 = 0.644$

Micro Recall: $65 / 151 = 0.430$

Interpretation: Queries with more relevant documents (Q2 and Q3) dominate the evaluation, so missing Q1 has a smaller effect.

- **Takeaways:**

- Macro: Use when each query matters equally; highlights worst-case or low-volume queries.
- Micro: Use when overall effectiveness is key; reflects impact of queries with more relevant documents.

3.3 Retrieval with Order

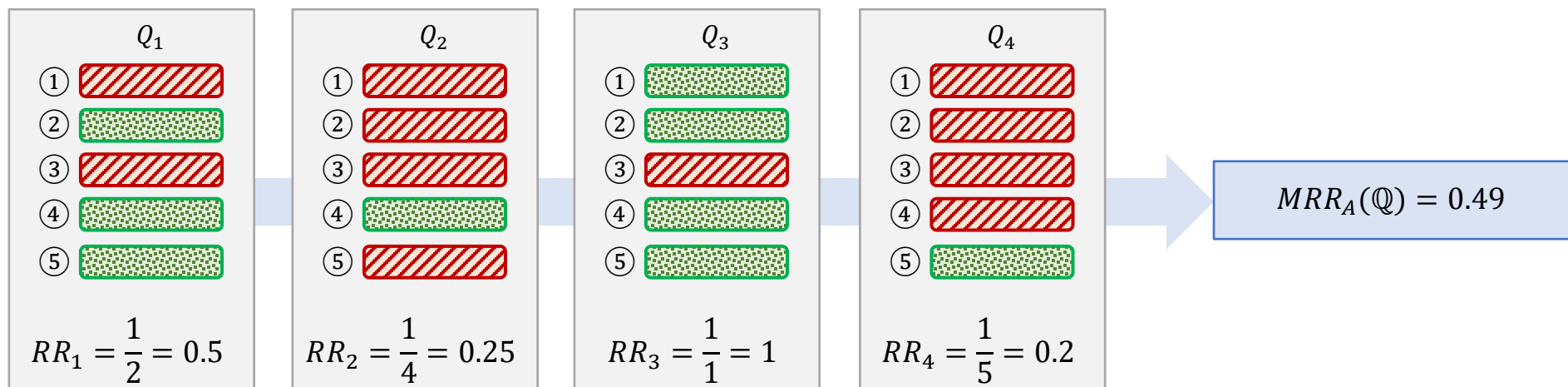
- In many retrieval scenarios, the order of results is crucial. For example, for web search engines or fact-checking tasks, users expect the answer to appear among the top results. In such cases, the **Mean Reciprocal Rank (MRR)** is the preferred metric. MRR is especially useful when dealing with benchmarks that have sparse assessments as it prevents meaningful calculations of precision and recall values. The definition for queries $Q_i \in \mathbb{Q}$ is as follows:

$$MRR(\mathbb{Q}) = \frac{1}{|\mathbb{Q}|} \sum_{Q_i \in \mathbb{Q}} \frac{1}{rank_i}$$

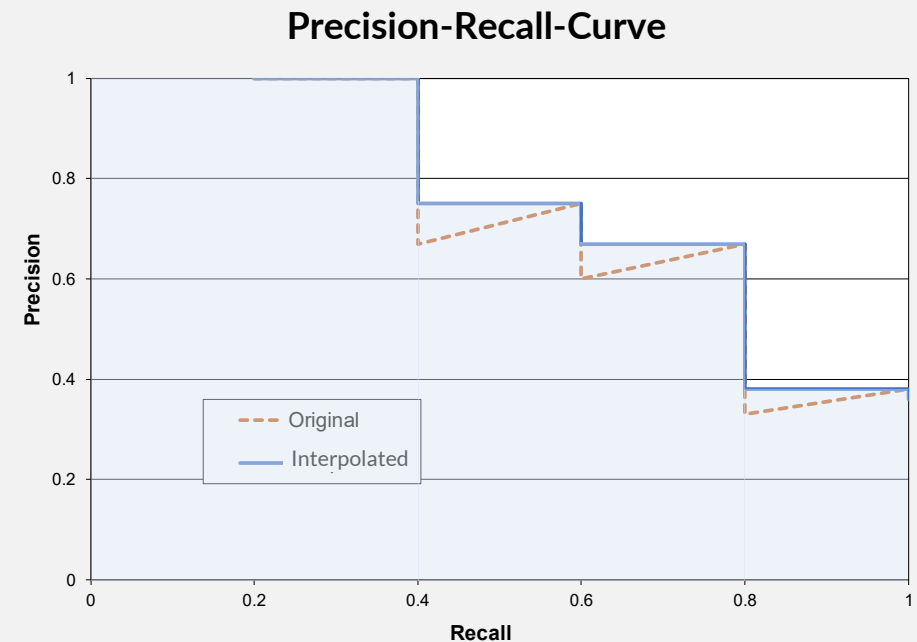
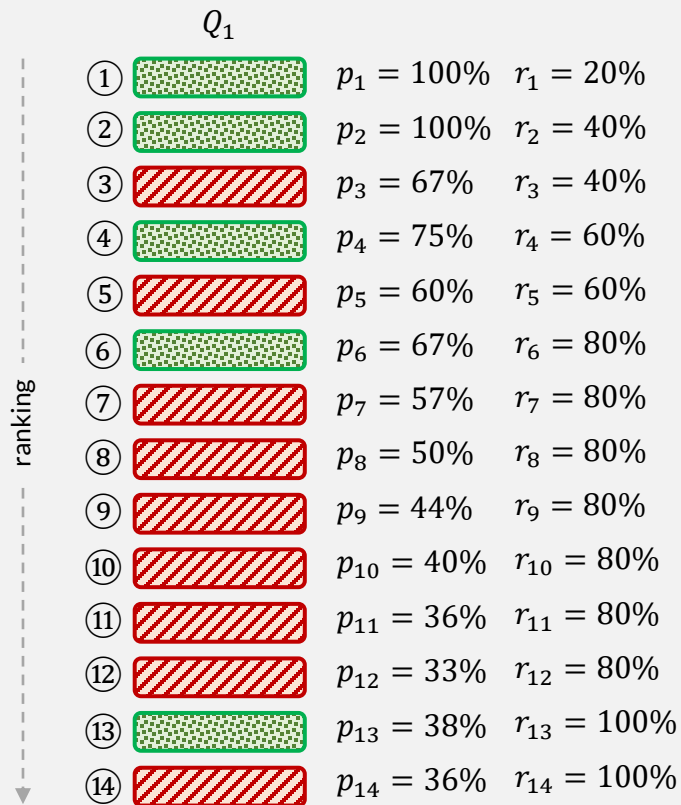
with $rank_i$ being the rank of the first relevant document for query Q_i . Unlike precision and recall, MRR only focuses on the first relevant document and ignores the rest. It places a high importance on the top position in the ranking and assigns lower importance to later positions, converging quickly to 0 as the rank increases.

- Example: consider the search method A and its results for queries Q_1 to Q_4 , as shown in the visualization below. The Mean Reciprocal Rank (MRR) is calculated as the average of the reciprocal ranks RR_i for all the queries Q_i .

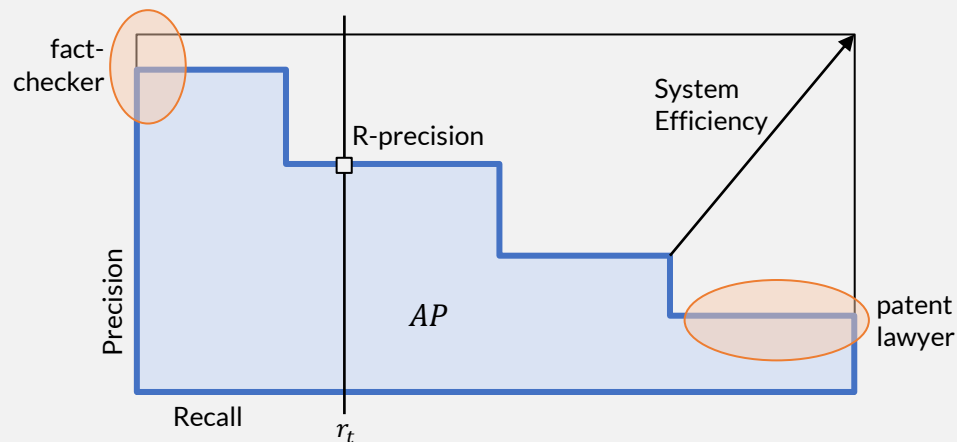
Search Method A



- We can extend the definition of precision and recall to include the ranking of retrieved documents. The **Precision-Recall Curve** only considers the top- k results in the ranking for $k = 1..n$ (as shown in the picture below on the left side). For each top- k result, it calculates the precision and recall values based on their relevance assessment. In the example below with $k = 3$, the precision is $p_3 = 2/3$, as 2 out of 3 documents in the top-3 are relevant. If we have 5 relevant documents overall in the collection, then the recall is $r_3 = 2/5$, as the top-3 contains 2 out of 5 relevant documents. We can calculate all the other precision p_i and recall r_i values, forming the precision-recall curve as depicted on the right side below.
- As we increase k , the precision increases when the next document is relevant and decreases if it is not relevant. On the other hand, recall values only increase whenever we find a new relevant document. This results in a characteristic "sawtooth" plot, which is often interpolated to simplify subsequent calculations such as the area under the precision-recall curve (blue area in the picture on the right side)



- Note that we can always achieve a recall value of 1 if we keep enumerating documents in the list until all relevant ones have been returned. Once we have all the relevant documents, the PR-curve becomes complete, and we can use it directly for visual comparisons between two methods or calculate simpler metrics for the comparison.
 - Near the point where recall is 0 and precision is 1, the PR-curve shows how well a method can answer fact-checker type queries where high precision for the top-10 documents is expected but recall does not really matter.
 - Near recall values of 1, the PR-curve identifies methods that can find most of the relevant documents in the collection. High precision values are preferred as they indicate low overhead when going through the result list.
 - The ideal system would achieve a precision and recall value of 1. The **system efficiency** is measured by calculating the distance d of the PR-curve to this ideal point. The system efficiency E is then given by $E = 1 - d/\sqrt{2}$.
 - The **precision at k (P@k)** is a commonly used measure, calculated as the precision p_k of the top- k results. It is often used when we are not interested in all relevant documents and, thus, do not consider recall values.
 - Similarly, the **R-precision** measures the precision once a threshold recall value r_t is reached. For example, with $r_t = 20\%$, the metric evaluates the precision once 1/5th of the relevant documents were found. This method requires knowing the total number of relevant documents in the collection.
 - Finally, the **average precision (AP)** measures the area under the PR-curve (blue area, formula on the right side). High AP values indicate that a method maintains high precision as more and more relevant documents are found.
 - By iterating over a set of queries $\mathbb{Q}$, we can easily calculate the mean values for all the measures introduced above. The formula on the lower right side shows an example for the **mean average precision (MAP)**.



$$AP = \int_0^1 p(r) dr = \sum_{k=1}^n p_k (r_k - r_{k-1})$$

$$MAP(\mathbb{Q}) = \frac{1}{|\mathbb{Q}|} \sum_{Q_i \in \mathbb{Q}} AP(Q_i)$$

3.4 Retrieval with Graded Relevance

- Until now, we have only considered binary relevance assessments for documents. However, we can also use graded relevance assessment where the grade indicates varying degrees of match between the document and the query. For example, we can introduce relevance values between 0 and 3, where 0 means “not relevant”, 1 means “somewhat relevant”, 2 means “relevant”, and 3 means “highly relevant”. The grades now influence how we assess a search method: higher grades are preferred over lower grades at the top of the rankings.
- The **cumulative gain (CG-k)** is a measure of how valuable the top- k results are, similar to precision at k :

$$CG_k = \sum_{i=1}^k rel_i \qquad \widehat{CG}_k = \frac{\sum_{i=1}^k rel_i}{k \cdot rel_{max}} \qquad rel_i \in [0, rel_{max}]$$

with rel_i denoting the graded relevance of the document at position i . To obtain a value between 0 and 1, CG_i needs to be normalized with $k \cdot rel_{max}$. In the case of binary relevance assessments, $\widehat{CG}_k$ is equal to precision at k .

- The **discounted cumulative gain (DCG-k)** incorporates the ranking by penalizing relevant documents in lower ranks:

$$DCG_k = \sum_{i=1}^k \frac{rel_i}{\log_2(i+1)} \qquad \text{variant: } DCG_k = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i+1)}$$

The variant on the right emphasizes relevant documents more strongly than the left formula. Other variants may use different logarithm bases and slight modifications of this approach.

- Search results for a given query may have varying lengths, making it challenging to interpret DCG-k values and compare them across queries. To address this, we compute a **normalized DCG (nDCG-k)** by first establishing an ideal ranking where documents are sorted by their graded relevance in descending order, and then calculating the DCG-k value for this ideal ranking. This yields the highest possible value, known as the ideal DCG-k (IDCG-k), which we use to normalize the DCG value of the actual result.

$$nDCG_k = \frac{DCG_k}{IDCG_k} \qquad \text{with: } IDCG_k = \max DCG_k \quad (\text{over all possible rankings of documents})$$

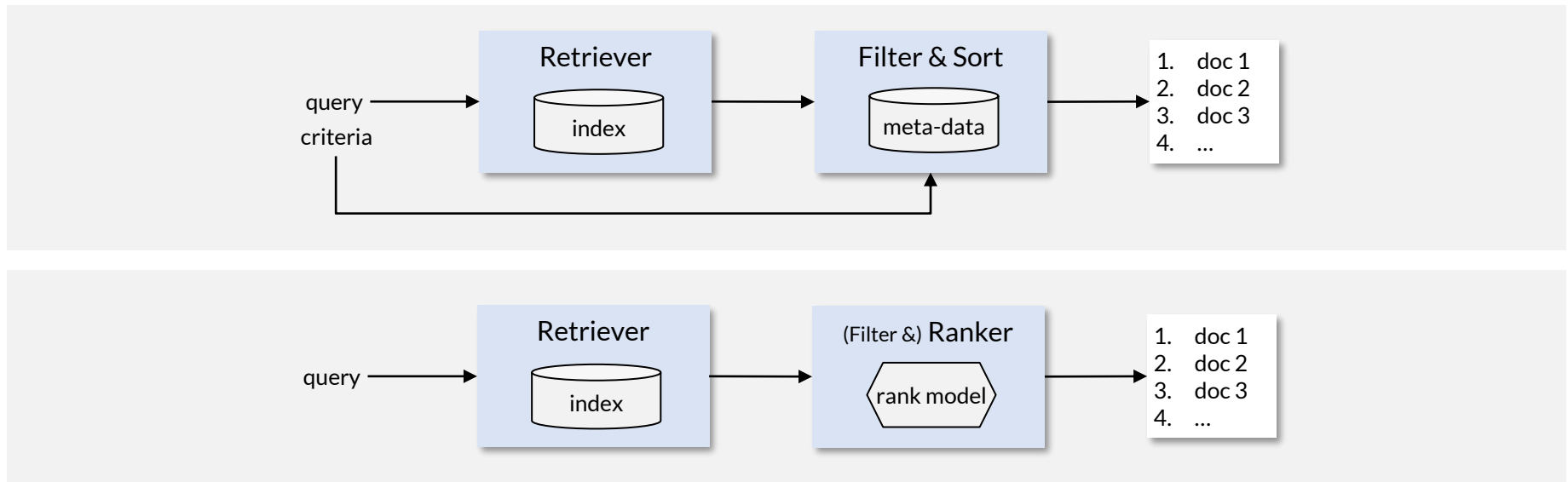
- Example: let's consider the first 10 documents of a search result with graded relevance values between 0 and 3

k	rel_k	CG_k	$\widehat{CG}_k$	$\frac{1}{\log_2(k+1)}$	DCG_k	ideal rel_k	$IDCG_k$	$nDCG_k$
1	0	0	0.00	1.00	0.00	3	3.00	0.00
2	2	2	0.33	0.63	1.26	3	4.89	0.26
3	1	3	0.33	0.50	1.76	3	6.39	0.28
4	3	6	0.50	0.43	3.05	3	7.68	0.40
5	0	6	0.40	0.38	3.05	3	8.85	0.35
6	2	8	0.44	0.36	3.77	2	9.56	0.39
7	0	8	0.38	0.33	3.77	2	10.22	0.37
8	3	11	0.46	0.32	4.71	2	10.86	0.43
9	1	12	0.44	0.30	5.01	2	11.46	0.44
10	3	15	0.50	0.29	5.88	2	12.04	0.49

- The table above displays 10 results, with the 2nd column indicating the graded relevance rel_k for each document at position k . The cumulative gain CG_k is the sum of these values up to position k . To obtain the normalized version, we divide CG_k by $3 \cdot k$. Consequently, we obtain $\widehat{CG}_{10} = 0.5$. For comparison, if we consider any $rel_k > 0$ as relevant, then the "precision at 10" is 0.70 overrating the documents with low relevance grades.
- The discounted cumulative gain DCG_k is shown in the next columns: first, we have the discount factors for each ranking position. By multiplying them with the graded relevance rel_k and summing them up to position k , we obtain the DCG_k values. Since we have not normalized the relevance values, they are difficult to interpret.
- For the normalized DCG, we assume there are a total of 5 documents with a graded relevance of 3 and 10 documents with a graded relevance of 2 in the collection. The "ideal rel_k " column shows an ideal ranking for this scenario, allowing us to compute the ideal DCG ($IDCG_k$) values and use them to obtain the normalized DCG ($nDCG_k$) values in the rightmost column. An $nDCG_{10}$ value of 0.49 illustrates solid performance in this example.

3.5 Retriever and Filter/Ranker

- In a previous chapter, we introduced two common retrieval architectures: **Retriever-Filter** and **Retriever-Ranker** systems. These patterns show how information retrieval systems choose and present relevant documents to users and differ in how they balance coverage and ordering. Understanding how they affect precision and recall is important for building effective systems:



- In both architectures the **retriever** first selects a broad set of candidate documents from the full collection. It aims for **high recall** so that most or all relevant documents are included. Precision at this stage is often lower because many irrelevant documents may also be retrieved. The goal is to avoid missing relevant documents and to shrink the pool of candidates that will be shown to the user.
- In **Retriever-Filter** systems, users apply filters like year, rating, or price to narrow results. This faceted search removes irrelevant items and improves precision. Because the feedback loop can shift which criteria are prioritized, the top results may not always be the most relevant.
- In **Retriever-Ranker** systems, the ranker gives each candidate document a relevance score. This step mainly improves **precision** at the top of the ranked list, without manual intervention of the user. Recall depends mostly on the retriever, but relevant documents ranked low can be missed if the user considers only the top results.

3.6 Literature and Links

- **MS MARCO** (Microsoft Machine Reading Comprehension Dataset), <https://microsoft.github.io/msmarco/>
- **Text REtrieval Conference**, <http://trec.nist.gov/>
- **Kaggle Competitions**, <https://www.kaggle.com/competitions>
- Kent, Allen; Berry, Madeline M.; Luehrs, Jr., Fred U.; Perry, J.W. (1955). **Machine literature searching VIII. Operational criteria for designing information retrieval systems**. American Documentation. 6 (2): 93. doi:[10.1002/asi.5090060209](https://doi.org/10.1002/asi.5090060209).
- Baeza-Yates, Ricardo; Ribeiro-Neto, Berthier (1999). **Modern Information Retrieval**. New York, NY: ACM Press, Addison-Wesley. [ISBN 0-201-39829-X](https://www.amazon.com/dp/020139829X)
- David A. Grossman, Ophir Frieder, **Information Retrieval Algorithms and Heuristics**, Kluwer Academic Publishers, 1998
- Christopher D. Manning, Prabhakar Raghavan and Hinrich Schütze, **Introduction to Information Retrieval, Chapter 8: Evaluation in information retrieval**, Cambridge University Press. 2008. Online version: <https://nlp.stanford.edu/IR-book/>
- D. R. Radev; H. Qi; H. Wu; W. Fan (2002). **Evaluating web-based question answering systems**, Proceedings of LREC. <http://www.lrec-conf.org/proceedings/lrec2002/pdf/301.pdf>
- Related Wikipedia articles
 - **Evaluation measures**, [https://en.wikipedia.org/wiki/Evaluation_measures_\(information_retrieval\)](https://en.wikipedia.org/wiki/Evaluation_measures_(information_retrieval))
 - **Discounted cumulative gain**, https://en.wikipedia.org/wiki/Discounted_cumulative_gain